Programmentwurf – PokerSim

Advanced Software Engineering / Programmentwurf

Studiengang / Kurs: TINF23

Dennis Gordeev
Matrikel: 4328420

Tim Haug
Matrikel: 1938743

Nico Reimer
Matrikel: 5446189

Abgabedatum: 03.05.2026

Repository: https://github.com/gordi76/PokerSim

Technologien: Java 17, Gradle, CLI-Anwendung

Inhaltsverzeichnis

1 Einführung ... 5
1.1 Übersicht über die Applikation ... 5
1.2 Starten der Applikation ... 5
1.2.1 Voraussetzungen ... 5
1.2.2 Anwendung starten ... 5
1.2.3 Tests ausführen ... 5
1.2.4 Beispiel-Spielablauf ... 5
1.3 Technischer Überblick ... 6
1.3.1 Verwendete Technologien ... 6
1.3.2 Projektstruktur ... 6
2 Softwarearchitektur ... 7
2.1 Clean Architecture ... 7
2.2 Domain Code ... 7
2.3 Dependency Rule ... 9
2.3.1 Positives Beispiel – `Deck` hängt von `RandomSource` ab ... 9
2.3.2 Negatives Beispiel – `GameRepository` im falschen Paket ... 10
3 SOLID ... 11
3.1 Single Responsibility Principle (SRP) ... 11
3.1.1 Positives Beispiel – `TexasHoldemHandEvaluator` ... 11
3.1.2 Negatives Beispiel – `PokerCli` ... 11
3.2 Open/Closed Principle (OCP) ... 12
3.2.1 Positives Beispiel – `HandEvaluator`-Interface ... 12
3.2.2 Negatives Beispiel – `PokerCli.handle()` ... 13
3.3 Dependency Inversion Principle (DIP) ... 13
3.3.1 Positives Beispiel – `Deck` hängt von `RandomSource` ... 13
3.3.2 Negatives Beispiel – `GameRepository` im falschen Paket ... 14
3.3.3 Lösung des Negativ-Beispiels ... 15
4 Weitere Prinzipien ... 16
4.1 GRASP: Geringe Kopplung ... 16
4.1.1 Beispiel – `GamePresenter` entkoppelt CLI von Domain ... 16
4.2 GRASP: Polymorphismus ... 17
4.2.1 Beispiel – `RandomSource` mit `JavaRandomSource` und `FixedRandomSource` ... 17
4.3 DRY ... 18
4.3.1 Beispiel – `GameRules` als Single Source of Truth ... 18
4.3.2 Beispiel – `formatCards` in `GamePresenter` ... 18
5 Unit Tests ... 19
5.1 10 Unit Tests ... 19
5.1.1 Codebeispiel: HandEvaluator-Tests ... 19
5.2 ATRIP ... 20
5.3 Fakes und Mocks ... 20
5.3.1 `FixedRandomSource` ... 20
5.3.2 `FakeGameRepository` ... 21
6 Domain Driven Design ... 24
6.1 Ubiquitous Language ... 24
6.2 Repository ... 24
6.3 Aggregate ... 25
6.4 Entity ... 26

6.5 Value Object ... 27
6.5.1 `Card` ... 27
6.5.2 `Chips` ... 28
7 Refactoring ... 29
7.1 Code Smells ... 29
7.1.1 Code Smell 1: Switch Statement in `PokerCli.handle()` ... 29
7.1.2 Code Smell 2: `PokerCli` als God Class ... 29
7.2 Refactorings ... 29
7.2.1 Refactoring 1: Introduce Interface – `RandomSource` ... 29
7.2.2 Refactoring 2: Extract Class – `GameRules` ... 30
7.2.3 Refactoring 3: Move Class – `GameRepository` ... 32
8 Entwurfsmuster ... 34
8.1 Repository Pattern ... 34
8.1.1 Beschreibung ... 34
8.1.2 Umsetzung in PokerSim ... 34
8.1.3 Begründung ... 34
8.2 Strategy Pattern ... 35
8.2.1 Beschreibung ... 35
8.2.2 Umsetzung in PokerSim ... 35
8.2.3 Begründung ... 36

1 Einführung

1.1 Übersicht über die Applikation

PokerSim ist eine textbasierte Java-CLI-Anwendung, die eine vereinfachte Runde Texas Hold'em Poker simuliert. Die Anwendung läuft vollständig im Terminal und verzichtet bewusst auf eine grafische Oberfläche, da die Aufgabenstellung ausdrücklich eine textbasierte Ausgabe verlangt.

Nutzer können ein Spiel mit beliebigen Spielernamen starten, die einzelnen Phasen einer Poker-Runde durchlaufen (Pre-Flop, Flop, Turn, River, Showdown), Karten anzeigen lassen, Einsätze setzen, folden und schließlich den Showdown auslösen, bei dem der beste Hand gewinnt.

Jedes Spiel wird nach jeder Aktion automatisch in eine Datei (`build/pokersim-game.txt`) persistiert, sodass der Spielstand auch nach einem Neustart wiederhergestellt werden kann.

1.2 Starten der Applikation

1.2.1 Voraussetzungen

- Java 17 oder neuer
- Git (zum Klonen des Repositories)
- Kein separater Build-Schritt nötig – Gradle Wrapper ist enthalten

1.2.2 Anwendung starten

Unter Windows:

```
.\gradlew.bat run
```

Unter Linux / macOS:

```
./gradlew run
```

1.2.3 Tests ausführen

Unter Windows:

```
.\gradlew.bat clean test
```

Unter Linux / macOS:

```
./gradlew clean test
```

1.2.4 Beispiel-Spielablauf

Nach dem Start erscheint eine Eingabeaufforderung. Folgender Ablauf startet ein Spiel mit zwei Spielern und durchläuft alle Phasen bis zum Showdown:

```
start Alice Bob
next
next
next
next
showdown
exit
```

Verfügbare Befehle:

Befehl	Beschreibung
`start <Name...>`	Startet ein neues Spiel mit den angegebenen Spielernamen
`next`	Wechselt in die nächste Spielphase

`bet <Name> <Betrag>`	Setzt einen Einsatz für den genannten Spieler
`fold <Name>`	Spieler wirft seine Karten weg
`show`	Zeigt den aktuellen Spielstand an
`hand`	Zeigt die Handkarten des ersten aktiven Spielers
`showdown`	Wertet Hände aus und kürt den Gewinner
`history`	Zeigt den Aktionsverlauf der aktuellen Runde
`help`	Listet alle verfügbaren Befehle auf
`exit`	Beendet das Programm

1.3 Technischer Überblick

1.3.1 Verwendete Technologien

Technologie	Begründung
Java 17	Objektorientiert, stabile LTS-Version; Sealed Interfaces, Records und Pattern Matching vereinfachen domänennahe Modellierung
Gradle (Wrapper)	Reproduzierbarer Build ohne lokale Installation; `gradlew`/`gradlew.bat` gewährleistet plattformunabhängige Builds
JUnit 5	Standardframework für Unit-Tests in Java; unterstützt parametrisierte Tests und aussagekräftige `@DisplayName`-Annotationen
Git / GitHub	Versionskontrolle; Gruppenleistung über Commit-Historie nachvollziehbar
GitHub Actions	Automatischer Build und Testlauf bei jedem Push, läuft auf Ubuntu-Linux

1.3.2 Projektstruktur

```
src/
  main/java/de/pokersim/
    domain/        ← fachlicher Kern
    application/   ← Use Cases
    adapters/      ← Controller, Presenter, ViewModel
    cli/           ← Konsoleneinstieg
    infrastructure/← Persistenz, Zufallsquelle
  test/java/de/pokersim/
    ...            ← 21 Testklassen, >85 Tests
build.gradle
settings.gradle
gradlew / gradlew.bat
```

2 Softwarearchitektur

2.1 Clean Architecture

PokerSim ist nach dem Prinzip der **Clean Architecture** aufgebaut. Die Anwendung ist in fünf konzentrische Schichten gegliedert, wobei Abhängigkeiten immer nur nach innen zeigen dürfen – nie nach außen.

```
┌──────────────────────────────────────┐
│  CLI                                 │
│  ┌──────────────────────────────┐    │
│  │  Adapters                    │    │
│  │  ┌──────────────────────┐    │    │
│  │  │  Application         │    │    │
│  │  │  ┌──────────────┐    │    │    │
│  │  │  │  Domain      │    │    │    │
│  │  │  └──────────────┘    │    │    │
│  │  └──────────────────────┘    │    │
│  │        ▲  Infrastructure     │    │
│  └────────┼─────────────────────┘    │
└───────────┼──────────────────────────┘
            │ (Domain nutzt nur sich selbst)
```

Paket	Verantwortung
`domain`	Fachlicher Kern: Entitäten, Value Objects, Domänenregeln – keine Abhängigkeit zu Frameworks oder anderen Schichten
`application`	Use Cases, die Domänenoperationen orchestrieren und Persistenz über das `GameRepository`-Interface anstoßen
`adapters`	Controller, Presenter, ViewModel und CommandParser – übersetzen zwischen CLI-Welt und Application-Layer
`cli`	Einstiegspunkt: `Main` verdrahtet alle Abhängigkeiten; `PokerCli` steuert die Eingabeschleife
`infrastructure`	Konkrete Implementierungen für Persistenz (`FileGameRepository`, `InMemoryGameRepository`) und Zufall (`JavaRandomSource`)

2.2 Domain Code

Der Domain-Layer enthält den vollständigen fachlichen Kern und hat keinerlei Abhängigkeit zu Frameworks, Datenbanken oder der Konsole. Er ist dadurch unabhängig testbar und austauschbar.

Die zentralen Klassen im Überblick:

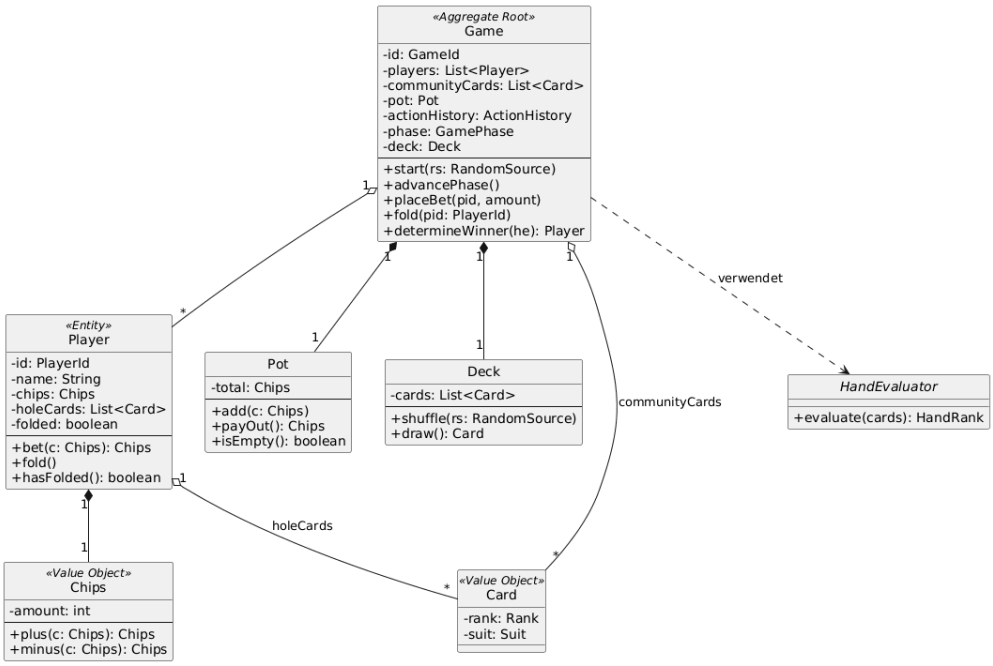

Abbildung 1: Überblick der wichtigsten Domain-Klassen

`Game` – Aggregate Root der Domäne. Koordiniert Spieler, Deck, Pot, ActionHistory und Spielphase. Jede Spielaktion (Starten, Phasenwechsel, Bet, Fold, Showdown) ist eine Methode auf `Game`.

```
public final class Game {
    private final GameId id;
    private final List<Player> players;
    private final List<Card> communityCards;
    private final Pot pot;
    private final ActionHistory actionHistory;
    private GamePhase phase;
    private Deck deck;

    public void start(RandomSource randomSource) { ... }
    public void advancePhase() { ... }
    public void placeBet(PlayerId playerId, Chips amount) { ... }
    public void fold(PlayerId playerId) { ... }
    public Player determineWinner(HandEvaluator evaluator) { ... }
}
```

`Player` – Entity mit fester Identität (`PlayerId`). Besitzt Name, Chips, Hole Cards und Fold-Status.

`Card` – Immutable Value Object aus `Rank` und `Suit`.

`Chips` – Immutable Value Object für Geldbeträge. Implementiert `Comparable<Chips>`, unterstützt `plus()` und `minus()`.

```
public final class Chips implements Comparable<Chips> {
    private final int amount;

    public Chips plus(Chips other)  { return new Chips(this.amount +
other.amount); }
    public Chips minus(Chips other) { return new Chips(this.amount -
other.amount); }
}
```

`TexasHoldemHandEvaluator` – implementiert `HandEvaluator` und bewertet eine 5-Karten-Hand nach Texas-Hold'em-Regeln (High Card bis Straight Flush inkl. Wheel-Straight A-2-3-4-5).

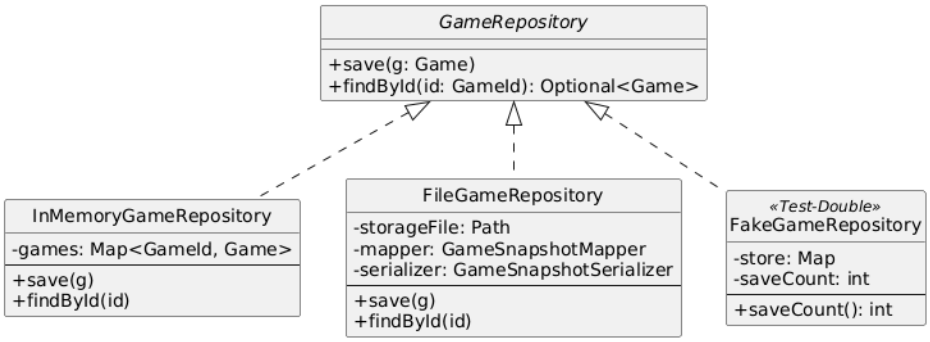

Abbildung 2: Repository Pattern: `GameRepository` mit Persistenz-Abstrahierung

`GameRules` – Utility-Klasse mit Spielkonstanten als Single Source of Truth:

```
public final class GameRules {
    public static final int INITIAL_CHIPS = 1_000;
    public static final int SMALL_BLIND   = 10;
    public static final int MIN_PLAYERS   = 2;
    public static final int MAX_PLAYERS   = 9;
    private GameRules() {}
}
```

2.3 Dependency Rule

Die Dependency Rule der Clean Architecture besagt: Quellcode-Abhängigkeiten dürfen immer nur nach innen zeigen – äußere Schichten dürfen innere kennen, aber nicht umgekehrt.

2.3.1 Positives Beispiel – `Deck` hängt von `RandomSource` ab

`Deck` (Domain-Layer) benötigt Zufälligkeit zum Mischen. Statt direkt `java.util.Random` zu verwenden, hängt es vom Interface `RandomSource` (Infrastructure-Layer-Interface) ab:

```
// Infrastructure-Interface
public interface RandomSource {
    int nextInt(int upperBoundExclusive);
}

// Domain nutzt das Interface – nicht die Implementierung
public final class Deck {
    public void shuffle(RandomSource randomSource) { ... }
}
```

Damit bleibt die Domäne von konkreter Java-API-Abhängigkeit entkoppelt und ist mit `FixedRandomSource` deterministisch testbar.

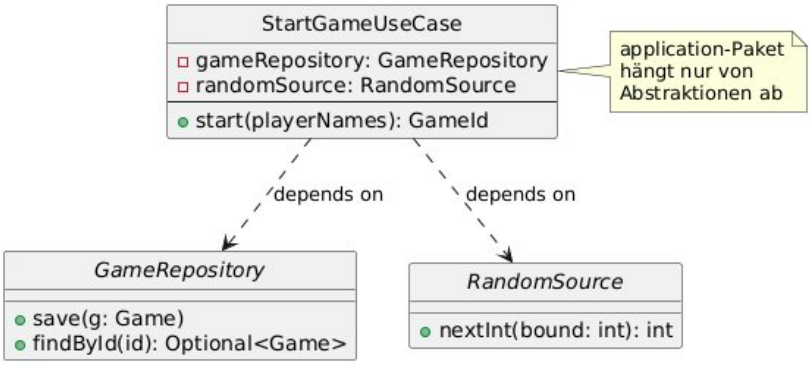

Abbildung 3: Positives Beispiel: `StartGameUseCase` hängt nur von Abstraktionen ab

2.3.2 Negatives Beispiel – `GameRepository` im falschen Paket

Das Interface `GameRepository` liegt im Paket `de.pokersim.infrastructure`. Application-Use-Cases importieren es von dort:

```
// StartGameUseCase.java (application-Layer)
import de.pokersim.infrastructure.GameRepository;  // ← Abhängigkeit nach außen
```

Nach strikter Clean Architecture sollte `GameRepository` im `application`- oder `domain`-Paket liegen, damit der Application-Layer nicht vom Infrastructure-Layer abhängt. Hier zeigt die Abhängigkeit in die falsche Richtung.

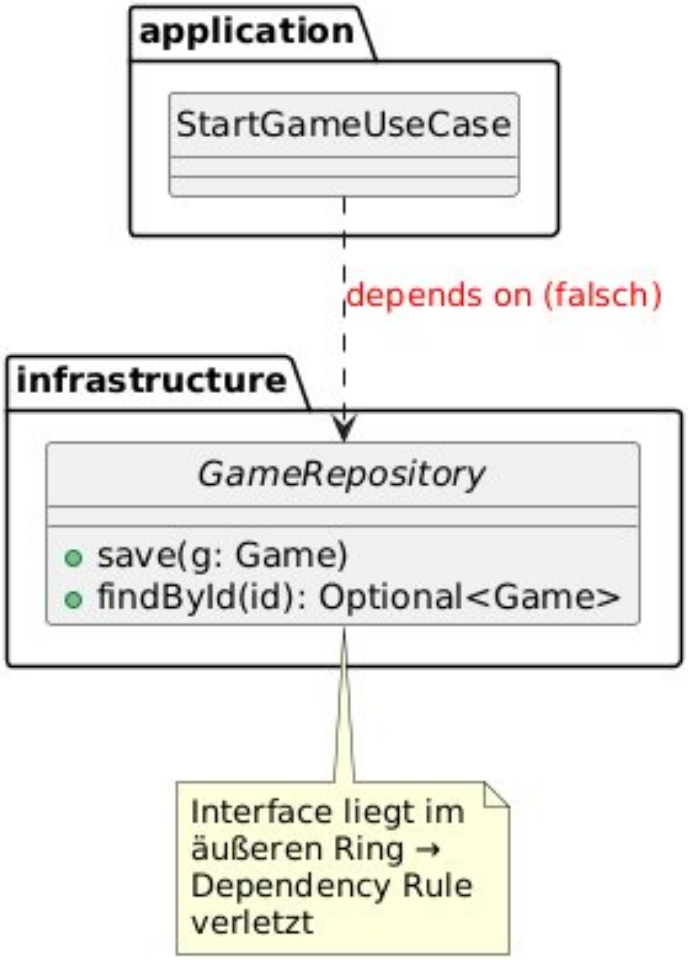

Abbildung 4: Negatives Beispiel: `application` zeigt auf `infrastructure` – Dependency Rule verletzt

3 SOLID

3.1 Single Responsibility Principle (SRP)

Das Single Responsibility Principle besagt, dass eine Klasse nur einen einzigen Grund zur Änderung haben soll – sie soll genau eine Verantwortung tragen.

3.1.1 Positives Beispiel – `TexasHoldemHandEvaluator`

`TexasHoldemHandEvaluator` hat genau eine Aufgabe: Eine Liste von Karten zu einem `HandRank` auswerten. Die Klasse enthält ausschließlich Bewertungslogik (Rang-Zählung, Flush-Erkennung, Straight-Prüfung). Sie kümmert sich weder um Ausgabe noch um Persistenz noch um Spielzustand.

```
public final class TexasHoldemHandEvaluator implements HandEvaluator {
    @Override
    public HandRank evaluate(List<Card> cards) {
        if (hasStraight(cards) && hasFlush(cards))  return
HandRank.STRAIGHT_FLUSH;
        if (containsCount(countRanks(cards), 4))     return
HandRank.FOUR_OF_A_KIND;
        // ...
    }
}
```

Ändert sich die Bewertungslogik (z. B. Joker-Regeln), ist ausschließlich diese eine Klasse betroffen.

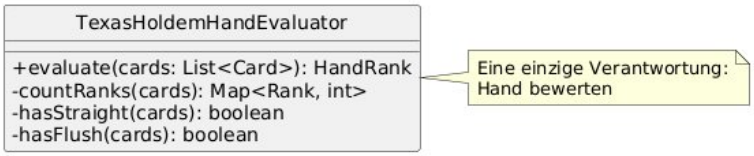

Abbildung 5: SRP positiv: `TexasHoldemHandEvaluator` mit nur einer Verantwortung

3.1.2 Negatives Beispiel – `PokerCli`

`PokerCli` verletzt das SRP, weil sie mehrere Verantwortlichkeiten in sich vereint:

- Programmschleife steuern (`run()`)
- Benutzereingaben lesen und parsen
- Konsolenausgabe formatieren
- Fehler abfangen und ausgeben
- Commands dispatchen (`handle()` mit switch)
- Spielregeln anzeigen (`printRules()`)

```
public final class PokerCli {
    public void run() { /* Hauptschleife */ }
    private boolean handle(ParsedCommand command) {
        return switch (command.name()) {
            case "start"    -> { ... yield true; }
            case "next"     -> { ... yield true; }
            case "bet"      -> { ... yield true; }
            case "fold"     -> { ... yield true; }
            case "show"     -> { ... yield true; }
            case "showdown" -> { ... yield true; }
            case "help"     -> { ... yield true; }
            case "exit"     -> { ... yield false; }
            // ...
        };
    }
```

```
    private void printWelcome() { ... }
    private void printHelp()    { ... }
    private void printGame(...)  { ... }
    private void printRules()   { ... }
    private void printSummary() { ... }
}
```

Jede Änderung an Ausgabeformat, Fehlerbehandlung oder der Menge der Commands berührt dieselbe Klasse. Ein möglicher Lösungsansatz wäre das Command-Pattern: Jeder Befehl wird in eine eigene Klasse ausgelagert, die `handle()` implementiert. `PokerCli` würde nur noch die Schleife und das Dispatching übernehmen.

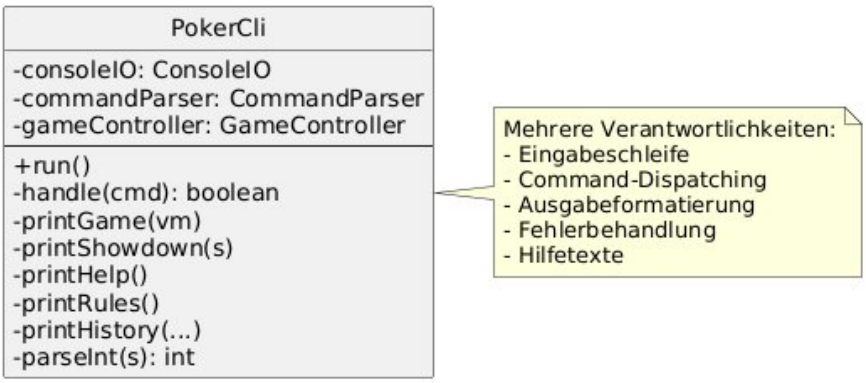

Abbildung 6: SRP negativ: `PokerCli` vereint mehrere Verantwortlichkeiten

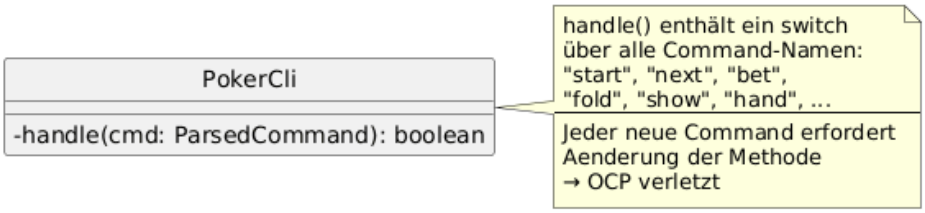

Abbildung 7: OCP negativ: `PokerCli.handle()` mit großer `switch`-Anweisung

3.2 Open/Closed Principle (OCP)

Das Open/Closed Principle besagt: Software-Einheiten sollen offen für Erweiterung, aber geschlossen für Modifikation sein. Neues Verhalten soll durch Hinzufügen von Code, nicht durch Ändern von vorhandenem Code entstehen.

3.2.1 Positives Beispiel – `HandEvaluator`-Interface

Das Interface `HandEvaluator` ist der Erweiterungspunkt für Handbewertungsstrategien. Um eine neue Variante (z. B. Omaha Hold'em) zu unterstützen, genügt eine neue Implementierung – bestehender Code bleibt unverändert:

```
public interface HandEvaluator {
    HandRank evaluate(List<Card> cards);
}

// Bestehend:
public final class TexasHoldemHandEvaluator implements HandEvaluator { ... }

// Erweiterung ohne Codeänderung:
// public final class OmahaHandEvaluator implements HandEvaluator { ... }
```

Ebenso gilt dies für `RandomSource`: neue Zufallsquellen können jederzeit hinzugefügt werden, ohne bestehenden Code zu ändern.

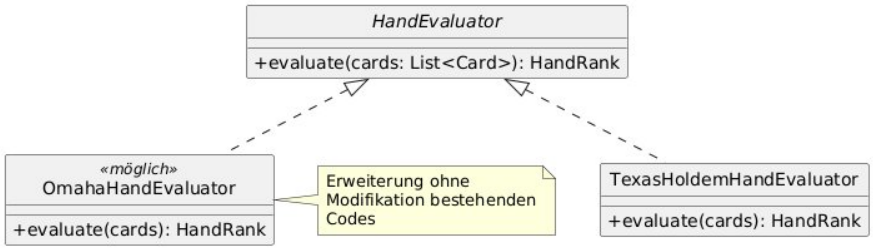

Abbildung 8: OCP positiv: `HandEvaluator` ist offen für neue Implementierungen

3.2.2 Negatives Beispiel – `PokerCli.handle()`

Die `handle()`-Methode in `PokerCli` enthält eine `switch`-Anweisung über Befehlsnamen. Jeder neue Befehl erfordert eine Änderung dieser Methode – das verletzt OCP:

```
private boolean handle(CommandParser.ParsedCommand command) {
    return switch (command.name()) {
        case "start"    -> { ... }
        case "next"     -> { ... }
        case "bet"      -> { ... }
        // neuer Command → Code muss hier geändert werden
        default -> { consoleIO.printLine("Unknown command."); yield true; }
    };
}
```

Lösungsidee: Command-Pattern mit einer `Map<String, CommandHandler>`, bei der neue Commands einfach registriert werden, ohne den Dispatch-Code anzufassen.

3.3 Dependency Inversion Principle (DIP)

Das Dependency Inversion Principle besagt: Hochrangige Module sollen nicht von niedrigrangigen Modulen abhängen – beide sollen von Abstraktionen abhängen.

3.3.1 Positives Beispiel – `Deck` hängt von `RandomSource`

`Deck` (Domain-Layer) hängt nicht von `java.util.Random` oder `JavaRandomSource` ab, sondern vom Interface `RandomSource`. Der Domain-Layer definiert damit, **welche Fähigkeit** er benötigt, ohne die konkrete Implementierung zu kennen:

```
// Interface in infrastructure (Abstraktion)
public interface RandomSource {
    int nextInt(int upperBoundExclusive);
}

// Domain hängt von der Abstraktion
public final class Deck {
    public void shuffle(RandomSource randomSource) {
        // Zufällige Permutation über randomSource.nextInt(...)
    }
}

// Konkrete Impl. in infrastructure
public final class JavaRandomSource implements RandomSource {
    private final Random random = new Random();
    @Override
    public int nextInt(int upperBoundExclusive) {
```

```
        return random.nextInt(upperBoundExclusive);
    }
}
```

Für Tests wird `FixedRandomSource` injiziert, die deterministisch arbeitet – ohne Änderung am Domain-Code.

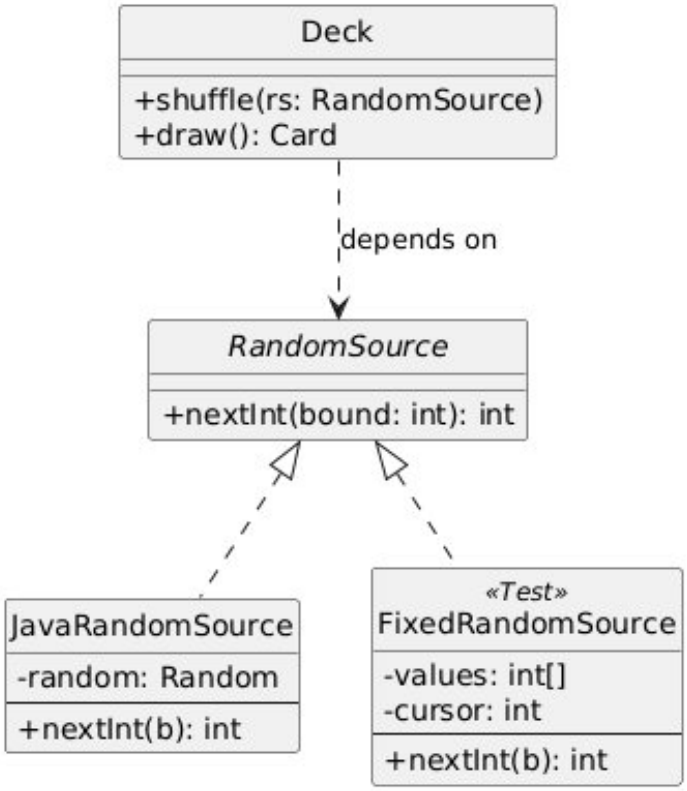

Abbildung 9: DIP positiv: `Deck` und konkrete `RandomSource`-Implementierungen hängen beide vom Interface ab

3.3.2 Negatives Beispiel – `GameRepository` im falschen Paket

`GameRepository` liegt in `de.pokersim.infrastructure`, obwohl es ein Interface ist, das von `application`-Use-Cases gebraucht wird. Der Application-Layer hängt damit direkt vom Infrastructure-Paket ab:

```
// StartGameUseCase.java
import de.pokersim.infrastructure.GameRepository; // ← Abhängigkeit in falsche
Richtung

public final class StartGameUseCase {
    private final GameRepository gameRepository; // Interface, aber im falschen
Paket
}
```

Korrekt wäre, `GameRepository` in `de.pokersim.application` oder `de.pokersim.domain` zu definieren, sodass Infrastructure das Interface implementiert – nicht umgekehrt.

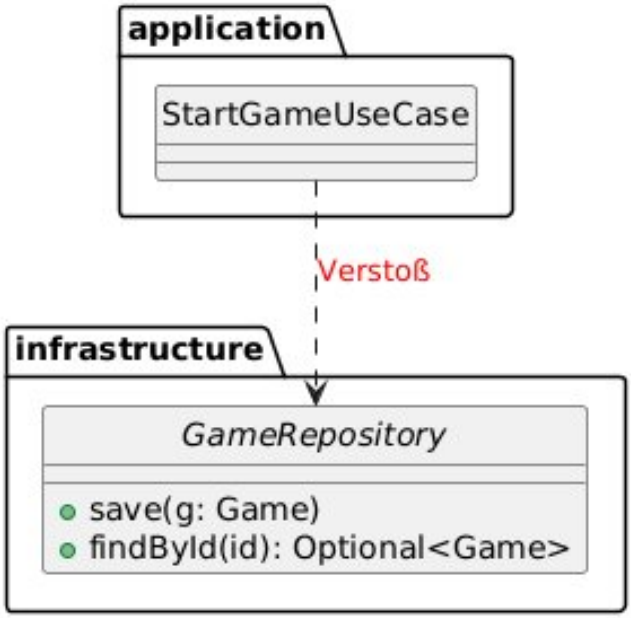

Abbildung 10: DIP negativ: `application` zeigt auf `infrastructure` – DIP verletzt

3.3.3 Lösung des Negativ-Beispiels

Das Interface `GameRepository` wird ins `application`-Paket verschoben (Refactoring „Move Class“). Dadurch implementiert `infrastructure` das Interface des `application`-Layers – die Abhängigkeit zeigt jetzt nach innen:

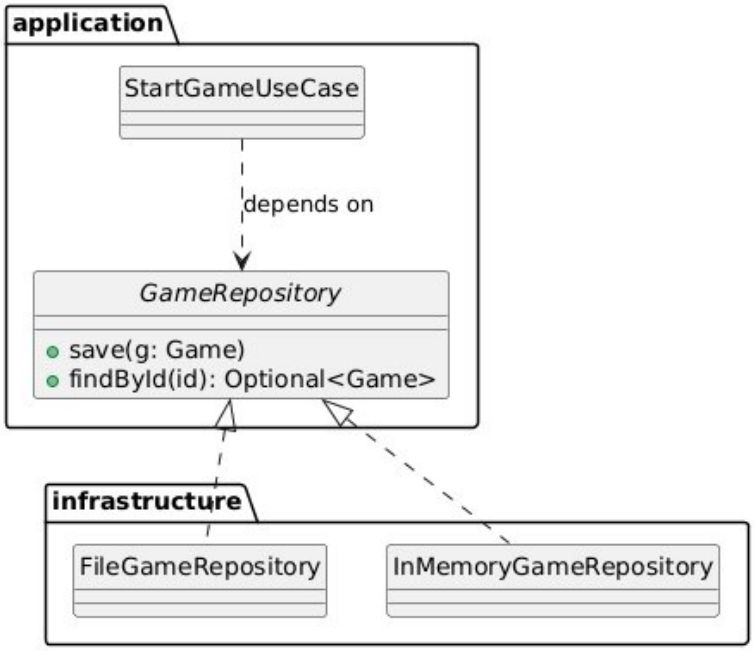

Abbildung 11: Lösung: `GameRepository` im `application`-Paket – Abhängigkeit zeigt nach innen

Nach diesem Refactoring importiert `StartGameUseCase` das Interface aus `de.pokersim.application`. `FileGameRepository` und `InMemoryGameRepository` bleiben in `infrastructure` und implementieren das Interface des inneren Rings. Diese Lösung wird in Kapitel 7 nochmal als „Move Class„-Refactoring behandelt.

4 Weitere Prinzipien

4.1 GRASP: Geringe Kopplung

Das GRASP-Prinzip **Low Coupling** (Geringe Kopplung) fordert, dass Klassen möglichst wenige direkte Abhängigkeiten zu anderen Klassen besitzen. Hohe Kopplung macht Code schwerer änderbar und testbar.

4.1.1 Beispiel – `GamePresenter` entkoppelt CLI von Domain

Ohne `GamePresenter` müsste `PokerCli` direkt auf `Game`, `Player`, `Card` und `Chips` zugreifen und diese selbst formatieren. Das würde die CLI-Klasse stark an den Domain-Layer koppeln.

`GamePresenter` übernimmt die Übersetzung von `Game` zu `GameViewModel`. Die CLI kennt danach nur noch das einfache View-Model:

```
// GamePresenter — übersetzt Domain → ViewModel
public final class GamePresenter {
    public GameViewModel present(Game game) {
        boolean revealHoleCards =
            game.phase() == GamePhase.SHOWDOWN || game.phase() ==
GamePhase.FINISHED;

        List<String> players = new ArrayList<>();
        for (Player player : game.players()) {
            String hand = revealHoleCards
                    ? String.join(" ", formatCards(player.holeCards()))
                    : "?? ??";
            String status = player.hasFolded() ? " [folded]" : "";
            players.add(player.name() + status
                    + "  |  chips: " + player.chips().amount()
                    + "  |  hand: " + hand);
        }
        return new GameViewModel(
            game.id().value(), game.phase().name(),
            players, formatCards(game.communityCards()),
            communityLabel(game), game.pot().toString()
        );
    }
}
```

`GameViewModel` enthält nur Strings und Listen von Strings. `PokerCli` hat damit keine Abhängigkeit zu `Game`, `Player` oder `Card` mehr – nur zu `GameViewModel`. Ändert sich die Domain-Klasse `Player`, muss nur `GamePresenter` angepasst werden, nicht die CLI.

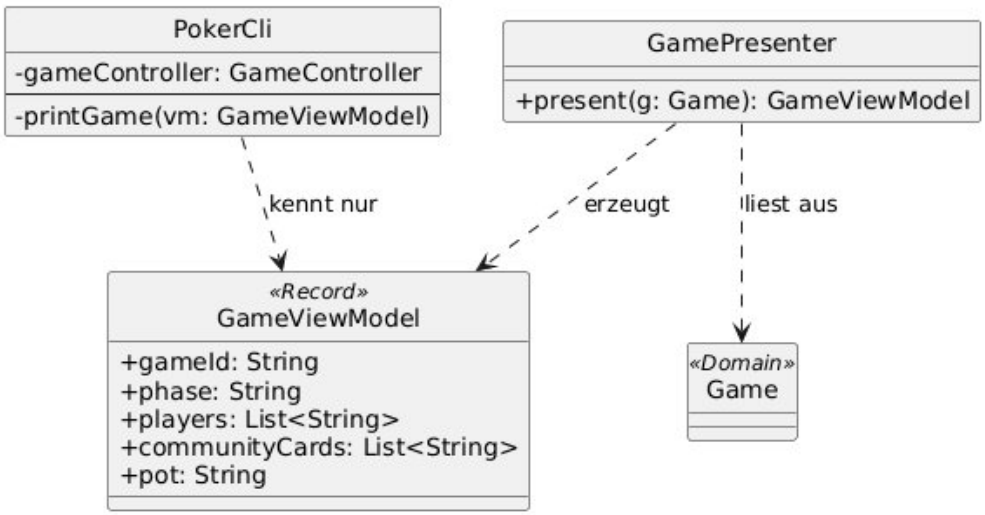

Abbildung 12: Geringe Kopplung: `GamePresenter` als Vermittler entkoppelt CLI von Domain

4.2 GRASP: Polymorphismus

Das GRASP-Prinzip **Polymorphismus** empfiehlt, variables Verhalten über Schnittstellen und deren Implementierungen zu realisieren, statt mit `if`/`switch`-Ketten auf Typinformationen zu prüfen.

4.2.1 Beispiel – `RandomSource` mit `JavaRandomSource` und `FixedRandomSource`

Statt im Code zu prüfen „Bin ich im Test-Modus?„, wird das variable Verhalten (echter Zufall vs. deterministischer Zufall) über Polymorphismus realisiert:

```
// Interface
public interface RandomSource {
    int nextInt(int upperBoundExclusive);
}

// Produktion: echter Java-Zufall
public final class JavaRandomSource implements RandomSource {
    private final Random random = new Random();
    @Override
    public int nextInt(int upperBoundExclusive) {
        return random.nextInt(upperBoundExclusive);
    }
}

// Test: deterministisch, kein echter Zufall
public final class FixedRandomSource implements RandomSource {
    private final int[] values;
    private int cursor;

    public FixedRandomSource(int... values) { this.values = values.clone(); }

    @Override
    public int nextInt(int upperBoundExclusive) {
        int raw = values[cursor % values.length];
        cursor++;
        return Math.floorMod(raw, upperBoundExclusive);
    }
}
```

`Deck.shuffle(RandomSource)` und `StartGameUseCase` nehmen das Interface entgegen. Welche Implementierung verwendet wird, bestimmt der Aufrufer – ohne Bedingungsverzweigungen im Domain-Code.

Gleiches gilt für `HandEvaluator` → `TexasHoldemHandEvaluator`: Die Bewertungslogik ist austauschbar, ohne dass Aufrufer angepasst werden müssen.

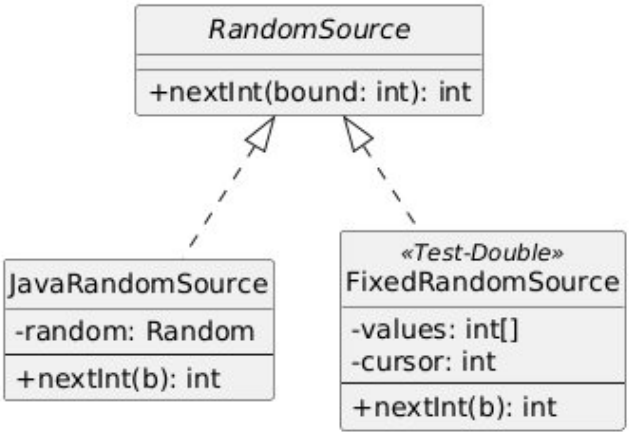

Abbildung 13: Polymorphismus: zwei Implementierungen, ein Aufruf-Punkt

4.3 DRY

Das Prinzip **Don't Repeat Yourself** (DRY) besagt, dass jedes Stück Wissen im System genau einmal vertreten sein soll. Doppelter Code erhöht den Wartungsaufwand, weil Änderungen an mehreren Stellen nachgezogen werden müssen.

4.3.1 Beispiel – `GameRules` als Single Source of Truth

Ursprünglich waren Spielkonstanten wie Startchips, Blinds und Spieleranzahl als Magic Numbers verstreut im Code. `GameRules` bündelt diese als benannte Konstanten:

```
// Vorher (dupliziert, fehleranfällig):
// game.addPlayer(name, new Chips(1000));  // in StartGameUseCase
// if (players.size() < 2) ...             // in Game
// actionHistory.record(..., new Chips(10)) // in Game (Small Blind)

// Nachher (DRY):
public final class GameRules {
    public static final int INITIAL_CHIPS = 1_000;
    public static final int SMALL_BLIND   = 10;
    public static final int MIN_PLAYERS   = 2;
    public static final int MAX_PLAYERS   = 9;
    private GameRules() {}
}
```

Jede Stelle im Code, die z. B. die Startchips benötigt, referenziert `GameRules.INITIAL_CHIPS`. Eine Regeländerung (z. B. andere Blind-Höhe) erfordert nur eine einzige Anpassung.

4.3.2 Beispiel – `formatCards` in `GamePresenter`

`GamePresenter` baut Kartenrepräsentationen an mehreren Stellen (Hole Cards, Community Cards). Die Formatierungslogik ist in private Methoden `formatCards()`, `formatCard()`, `shortRank()`, `shortSuit()` ausgelagert, statt dupliziert zu werden. Alle Stellen, die eine Karte anzeigen, nutzen dieselbe Methode.

5 Unit Tests

5.1 10 Unit Tests

Das Projekt enthält 21 Testklassen mit über 85 Unit-Tests, die alle fünf Architekturschichten abdecken. Im Folgenden werden 10 repräsentative Tests beschrieben.

Nr.	Testklasse	Beschreibung
1	`TexasHoldemHandEvaluatorTest`	Erkennt ein Paar: zwei Karten gleichen Rangs in einer 5-Karten-Hand ergeben `HandRank.PAIR`
2	`TexasHoldemHandEvaluatorTest`	Erkennt einen Flush: fünf Karten derselben Farbe ergeben `HandRank.FLUSH`
3	`TexasHoldemHandEvaluatorTest`	Erkennt eine Straight: fünf aufeinanderfolgende Ränge ergeben `HandRank.STRAIGHT`; Wheel-Straight (A-2-3-4-5) wird separat getestet
4	`GameTest`	Spiel startet korrekt: nach `start()` befindet sich das Spiel in Phase `PRE_FLOP` und alle Spieler haben zwei Hole Cards erhalten
5	`GameTest`	Phasenwechsel funktioniert: `advancePhase()` wechselt von `PRE_FLOP` zu `FLOP` und legt drei Community Cards auf den Tisch
6	`BettingRulesTest`	Gültiger Bet wird akzeptiert: ein Bet unter dem Chip-Bestand eines aktiven Spielers wirft keine Exception
7	`BettingRulesTest`	Ungültiger Bet wird verhindert: ein Bet, der den Chip-Bestand übersteigt, wirft `InvalidPlayerActionException`
8	`ActionHistoryTest`	ActionHistory speichert Aktionen: nach `record()` ist die Aktion über `actions()` abrufbar und filterbar nach Spieler und Phase
9	`FileGameRepositoryTest`	Repository schreibt Snapshot: nach `save()` existiert die Datei und enthält den Header `POKERSIM_GAME_SNAPSHOT`
10	`PokerCliTest`	CLI verarbeitet Start-Command: `start Alice Bob` führt dazu, dass das Spiel gestartet und der Spielstand ausgegeben wird

5.1.1 Codebeispiel: HandEvaluator-Tests

```java
class TexasHoldemHandEvaluatorTest {
    private final HandEvaluator evaluator = new TexasHoldemHandEvaluator();

    @Test
    @DisplayName("recognises a single pair")
    void recognisesPair() {
        List<Card> cards = List.of(
            new Card(Rank.TWO,  Suit.CLUBS),
            new Card(Rank.TWO,  Suit.HEARTS),
            new Card(Rank.SEVEN, Suit.DIAMONDS),
            new Card(Rank.NINE, Suit.SPADES),
            new Card(Rank.JACK, Suit.HEARTS)
        );
```

```
        assertEquals(HandRank.PAIR, evaluator.evaluate(cards));
    }

    @Test
    @DisplayName("recognises a straight")
    void recognisesStraight() {
        List<Card> cards = List.of(
            new Card(Rank.FOUR,  Suit.CLUBS),
            new Card(Rank.FIVE,  Suit.HEARTS),
            new Card(Rank.SIX,   Suit.DIAMONDS),
            new Card(Rank.SEVEN, Suit.SPADES),
            new Card(Rank.EIGHT, Suit.HEARTS)
        );
        assertEquals(HandRank.STRAIGHT, evaluator.evaluate(cards));
    }
}
```

5.2 ATRIP

Das ATRIP-Prinzip beschreibt die wesentlichen Eigenschaften guter Unit Tests: **Automatic**, **Thorough**, **Repeatable**, **Independent**, **Professional**.

Automatic – Tests laufen vollautomatisch ohne manuelle Eingriffe:

```
./gradlew clean test
```

GitHub Actions führt denselben Befehl bei jedem Push auf Ubuntu-Linux aus. Das Ergebnis ist ein HTML-Testbericht unter `build/reports/tests/test/index.html`.

Thorough – Alle Architekturschichten sind abgedeckt: Domain (8 Testklassen), Application (1), Adapters (3), CLI (1), Infrastructure (5). Tests prüfen sowohl Erfolgsfälle als auch Fehlerfälle (z. B. `InvalidPlayerActionException` bei ungültigem Bet).

Repeatable – Tests sind deterministisch. Überall, wo Zufälligkeit eine Rolle spielt, wird `FixedRandomSource` injiziert, sodass Kartenreihenfolgen vorhersehbar sind. `FileGameRepositoryTest` erstellt temporäre Dateien, die nach dem Test aufgeräumt werden.

Independent – Jeder Test legt seine Objekte in `@BeforeEach` neu an und teilt keinen Zustand mit anderen Tests. Die Testreihenfolge ist irrelevant.

Professional – Alle Tests tragen aussagekräftige `@DisplayName`-Annotationen auf Englisch. Testnamen beschreiben das erwartete Verhalten (z. B. `"recognises a single pair"`, `"rejects bet that exceeds chips"`). JUnit 5 wird konsistent mit `assertEquals`, `assertThrows` und `assertNotNull` verwendet.

5.3 Fakes und Mocks

Fakes sind vereinfachte Implementierungen echter Abhängigkeiten, die im Test statt der echten Implementierung eingesetzt werden. Sie ermöglichen isolierte, schnelle und deterministische Tests.

5.3.1 `FixedRandomSource`

Was ersetzt es? `JavaRandomSource` (echte `java.util.Random`-basierte Zufälligkeit).

Wer nutzt es? Tests für `Deck`, `Game`, `StartGameUseCase` und `GameService` – überall, wo das Mischen des Decks getestet wird.

Warum sinnvoll? Mit echtem Zufall wären Kartenreihenfolgen nicht vorhersehbar, Tests würden sporadisch fehlschlagen. `FixedRandomSource` liefert eine definierte Sequenz von Werten, sodass Testergebnisse exakt überprüfbar sind.

```java
public final class FixedRandomSource implements RandomSource {
    private final int[] values;
    private int cursor;

    public FixedRandomSource(int... values) { this.values = values.clone(); }

    public static FixedRandomSource noShuffle() {
        return new FixedRandomSource(0);
    }

    @Override
    public int nextInt(int upperBoundExclusive) {
        int raw = values[cursor % values.length];
        cursor++;
        return Math.floorMod(raw, upperBoundExclusive);
    }
}
```

Beziehung: `FixedRandomSource` und `JavaRandomSource` implementieren beide `RandomSource`. Tests injizieren `FixedRandomSource`; die Produktion nutzt `JavaRandomSource`.

5.3.2 `FakeGameRepository`

Was ersetzt es? `FileGameRepository` (schreibt auf Disk) und `InMemoryGameRepository`.

Wer nutzt es? Tests für `GameService`, `StartGameUseCase`, `AdvancePhaseUseCase` und alle weiteren Use-Case-Tests, die Persistenz benötigen.

Warum sinnvoll? `FileGameRepository` schreibt Dateien auf die Festplatte, was Tests langsamer und von Dateisystem-Zustand abhängig macht. `FakeGameRepository` hält alles im Speicher und zählt außerdem, wie oft `save()` und `findById()` aufgerufen wurden – damit lässt sich verifizieren, dass Use Cases korrekt persistieren.

```java
public final class FakeGameRepository implements GameRepository {
    private final Map<GameId, Game> store = new HashMap<>();
    private int saveCount;

    @Override
    public void save(Game game) {
        store.put(game.id(), game);
        saveCount++;
    }

    @Override
    public Optional<Game> findById(GameId id) {
        return Optional.ofNullable(store.get(id));
    }

    public int saveCount() { return saveCount; }
}
```

Beziehung: `FakeGameRepository`, `InMemoryGameRepository` und `FileGameRepository` implementieren alle `GameRepository`. Tests nutzen `FakeGameRepository`; Produktion nutzt `FileGameRepository`.

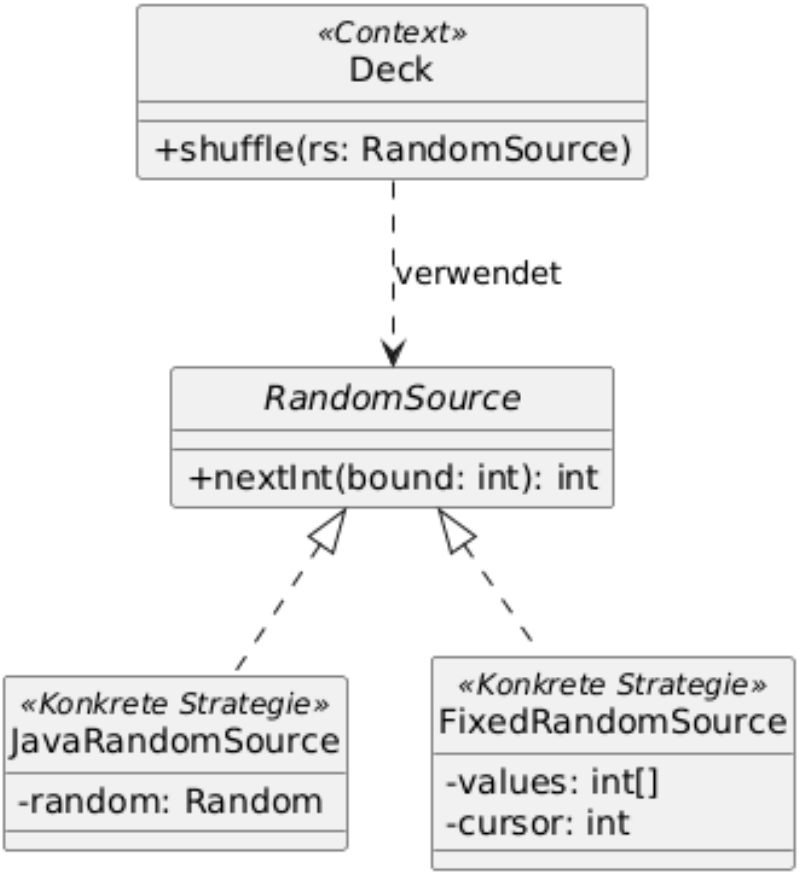

Abbildung 14: RandomSource Strategy Pattern für echte und Test-Implementierungen

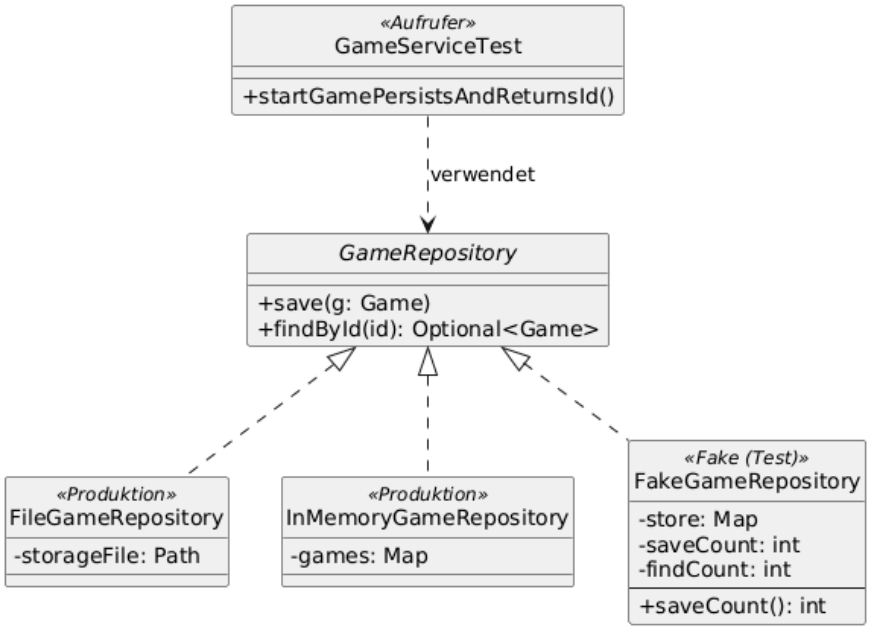

Abbildung 15: FakeGameRepository als Test-Double mit `saveCount()`-Zähler

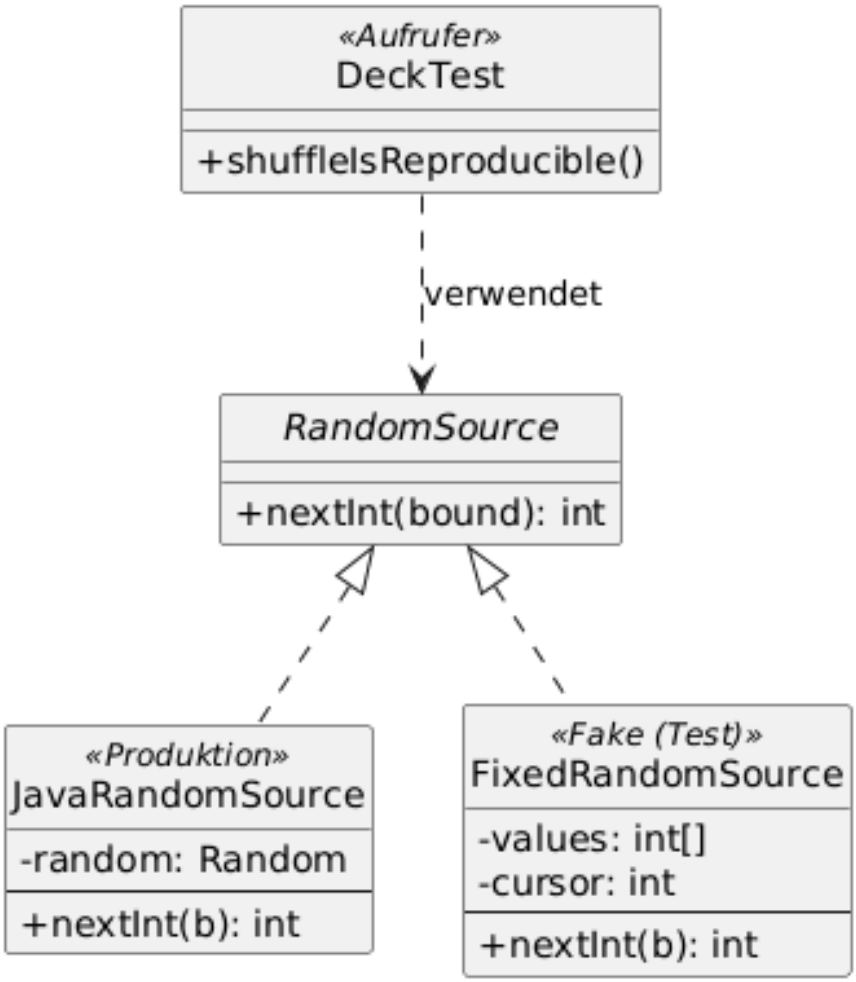

Abbildung 16: FixedRandomSource als Test-Double für deterministische Kartenreihenfolgen

6 Domain Driven Design

6.1 Ubiquitous Language

Domain Driven Design fordert eine gemeinsame Sprache (**Ubiquitous Language**) zwischen Entwicklern und Fachexperten. Diese Sprache spiegelt sich direkt im Code wider – Klassen-, Methoden- und Feldnamen entsprechen den Fachbegriffen.

Begriff	Bedeutung	Code-Stelle
Hole Cards	Die zwei verdeckten Privatkarten, die jeder Spieler zu Beginn erhält	`Player.holeCards`
Community Cards	Die gemeinsamen offenen Karten auf dem Tisch (Flop, Turn, River)	`Game.communityCards`
Pot	Der Gesamteinsatz aller Spieler in der aktuellen Runde	`Pot`, `Game.pot()`
Showdown	Die Schlussphase, in der Hände verglichen und der Gewinner ermittelt wird	`GamePhase.SHOWDOWN`
Flop	Die ersten drei Community Cards	`GamePhase.FLOP`, `GamePresenter.communityLabel()`
Big Blind / Small Blind	Obligatorische Vorabeinsätze der ersten beiden Spieler	`PlayerActionType.BIG_BLIND`, `GameRules.SMALL_BLIND`
Fold	Aufgeben: Spieler gibt seine Karten ab und verlässt die Runde	`PlayerActionType.FOLD`, `Player.fold()`

Alle Klassennamen, Enum-Werte und Methoden spiegeln exakt die Pokerterminologie wider. Es existiert keine technische Umbenennung (z. B. kein `UserCard` statt `Card`).

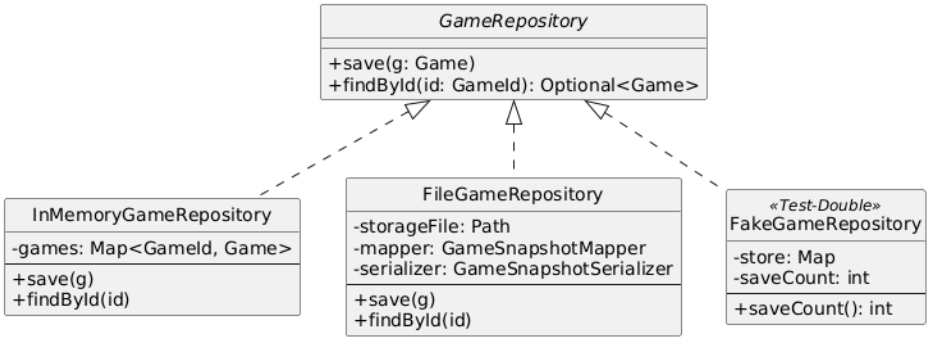

Abbildung 17: GameRepository Interface mit abstrahiertem Datenzugriff

6.2 Repository

Das Repository-Pattern abstrahiert den Datenzugriff hinter einer domänennahen Schnittstelle. Der Domain- und Application-Layer kennen keine Datenbankdetails oder Dateiformate.

```
public interface GameRepository {
    void save(Game game);
    Optional<Game> findById(GameId id);
}
```

Implementierungen:

- `InMemoryGameRepository` – speichert in einer `HashMap`; primär für Tests

- `FileGameRepository` – serialisiert den Spielstand als Text-Snapshot in `build/pokersim-game.txt`
- `FakeGameRepository` – Testdouble, das Aufrufzähler für `save()` und `findById()` führt

Die Produktion verwendet `FileGameRepository`, Tests verwenden `FakeGameRepository` oder `InMemoryGameRepository`. Der Application-Layer ist von dieser Entscheidung vollständig entkoppelt.

6.3 Aggregate

Ein **Aggregate** ist eine Gruppe von Domänenobjekten, die als konsistente Einheit behandelt werden. Es gibt genau eine **Aggregate Root**, über die alle Änderungen gehen.

In PokerSim ist `Game` das einzige Aggregate und dessen Aggregate Root:

```
public final class Game {
    private final GameId id;
    private final List<Player> players;   // Entities im Aggregate
    private final List<Card> communityCards;
    private final Pot pot;                // Value Object im Aggregate
    private final ActionHistory actionHistory;
    private GamePhase phase;
    private Deck deck;
}
```

`Player`-Objekte existieren nur im Kontext von `Game`. Zugriff auf Spieler erfolgt ausschließlich über `Game.players()` oder Operationen wie `Game.fold(PlayerId)`. Es gibt keinen direkten Zugriff auf `Player`-Instanzen am Repository vorbei. Der Zustand des Aggregates ist immer konsistent, da alle Änderungen durch `Game`-Methoden laufen, die Invarianten prüfen (z. B. über `BettingRules`).

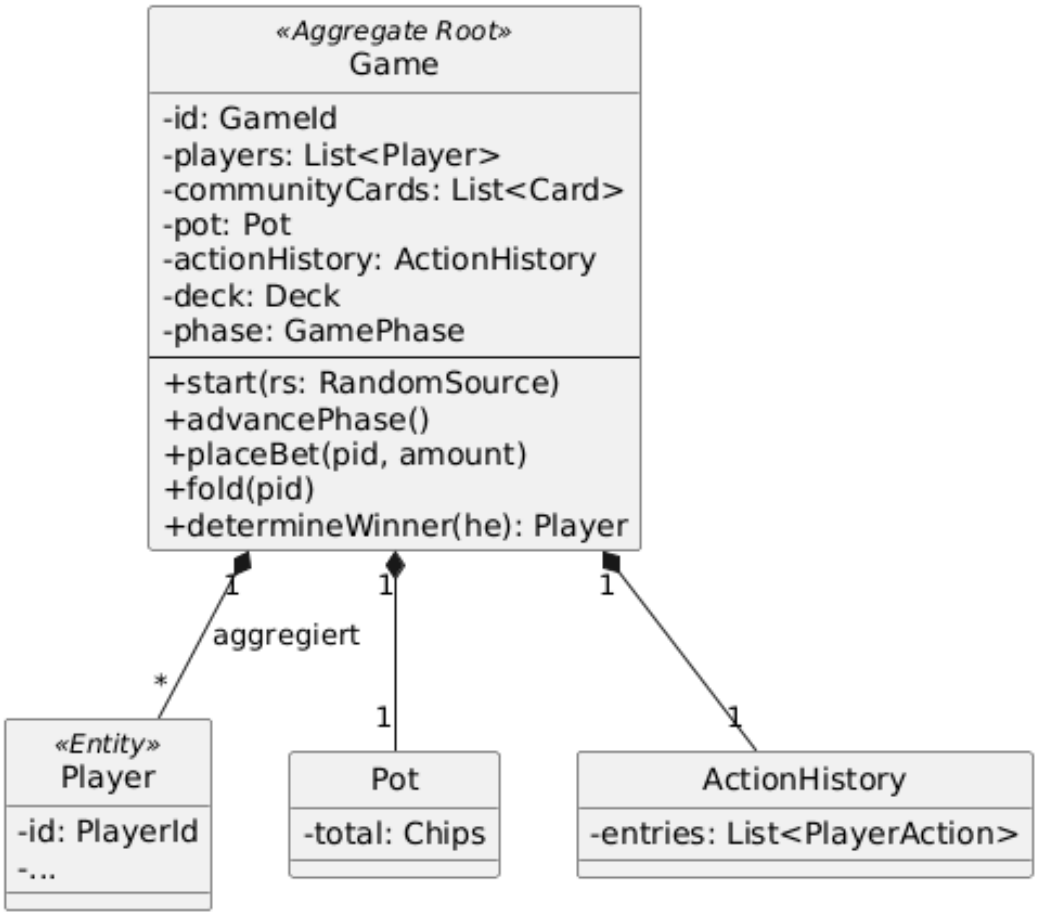

Abbildung 18: Game als Aggregate Root mit allen Komponenten

6.4 Entity

Eine **Entity** hat eine eindeutige Identität, die über die gesamte Lebensdauer stabil bleibt. Zwei Entities mit gleichen Attributen sind trotzdem verschieden, wenn ihre IDs unterschiedlich sind.

`Player` ist die zentrale Entity in PokerSim:

```
public final class Player {
    private final PlayerId id;     // stabile Identität
    private final String name;
    private Chips chips;           // veränderlich
    private final List<Card> holeCards;
    private boolean folded;        // veränderlich
}
```

Die Identität wird über `PlayerId` sichergestellt – ein Value Object, das eine UUID kapselt:

```
public final class PlayerId {
    private final String value;
    public static PlayerId newId() { return new
PlayerId(UUID.randomUUID().toString()); }
    public static PlayerId of(String value) { return new PlayerId(value); }
}
```

Zwei `Player`-Objekte mit gleichem Namen, aber unterschiedlichen `PlayerIds` sind unterschiedliche Spieler. Der Fold-Status und Chip-Bestand können sich ändern, die Identität bleibt konstant.

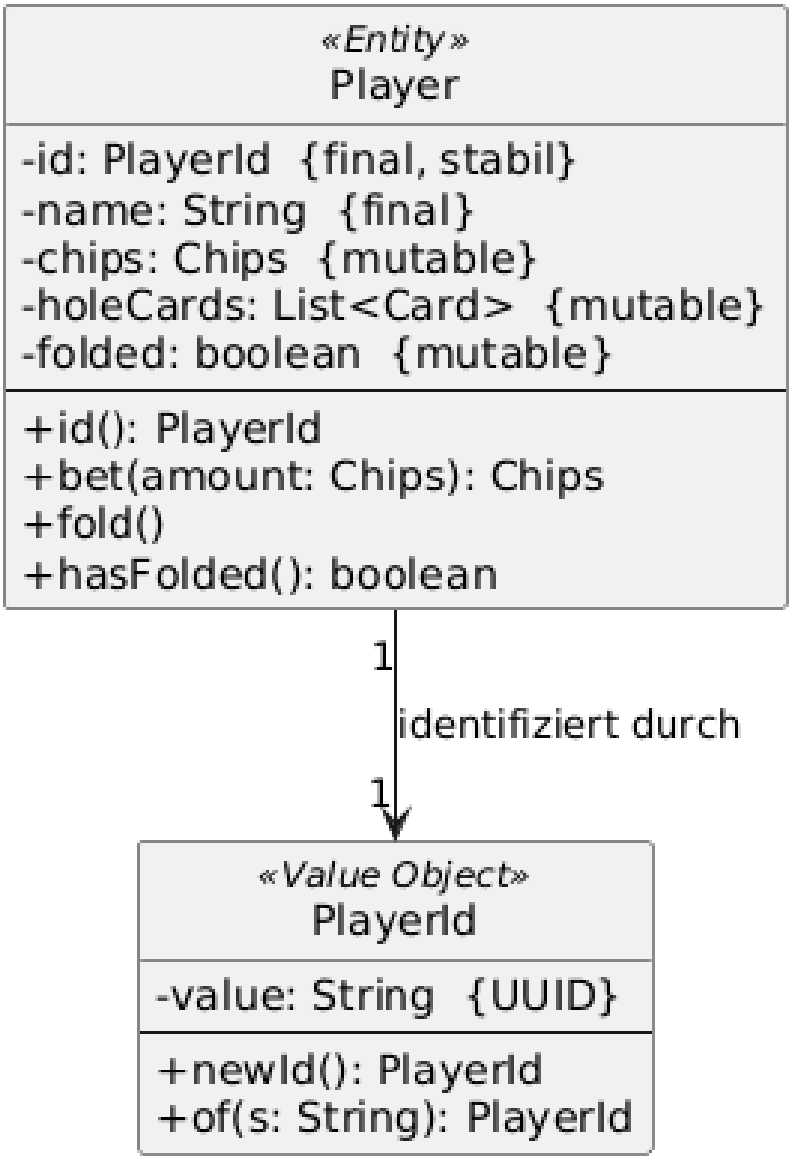

Abbildung 19: Entity Player mit Identität, Name und veränderlichen Attributen

6.5 Value Object

Ein **Value Object** hat keine eigene Identität – zwei Value Objects mit denselben Attributen sind gleich. Sie sind typischerweise immutable.

6.5.1 Card

```
public final class Card {
    private final Rank rank;
    private final Suit suit;

    @Override
    public boolean equals(Object other) {
        if (!(other instanceof Card card)) return false;
        return rank == card.rank && suit == card.suit;
    }

    @Override
```

```
    public int hashCode() { return Objects.hash(rank, suit); }
}
```

`Card` ist immutable (alle Felder `final`), hat keinen Identitäts-Anker und implementiert Wertegleichheit über `equals()`/`hashCode()`. Zwei `new Card(Rank.ACE, Suit.SPADES)` sind gleich.

6.5.2 Chips

```
public final class Chips implements Comparable<Chips> {
    private final int amount;

    public Chips plus(Chips other)  { return new Chips(this.amount +
other.amount); }
    public Chips minus(Chips other) { return new Chips(this.amount -
other.amount); }
    public boolean isZero()         { return amount == 0; }
}
```

`Chips` kapselt einen Geldbetrag. Operationen wie `plus()` und `minus()` geben neue Instanzen zurück – das Original wird nie verändert. Das verhindert unbeabsichtigte Seiteneffekte bei Berechnungen im Pot.

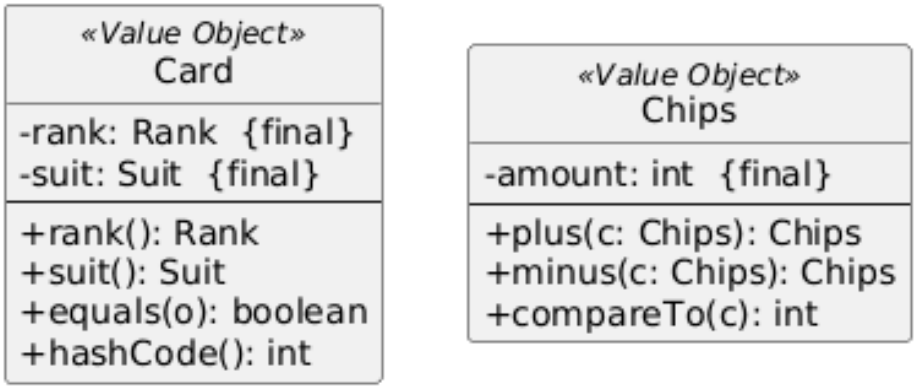

Abbildung 20: Value Objects Card und Chips mit Wertegleichheit

7 Refactoring

7.1 Code Smells

Code Smells sind Hinweise auf strukturelle Probleme im Code, die Wartung und Erweiterung erschweren, aber keine direkten Fehler darstellen.

7.1.1 Code Smell 1: Switch Statement in `PokerCli.handle()`

`PokerCli.handle()` enthält eine umfangreiche `switch`-Anweisung über Befehlsnamen:

```
private boolean handle(CommandParser.ParsedCommand command) {
    return switch (command.name()) {
        case "start"    -> { ... yield true; }
        case "next"     -> { ... yield true; }
        case "bet"      -> { ... yield true; }
        case "fold"     -> { ... yield true; }
        case "show"     -> { ... yield true; }
        case "hand"     -> { ... yield true; }
        case "showdown" -> { ... yield true; }
        case "history"  -> { ... yield true; }
        case "help"     -> { ... yield true; }
        case "exit"     -> false;
        default -> { consoleIO.printLine("Unknown command."); yield true; }
    };
}
```

Problem: Jeder neue Befehl erfordert eine Änderung dieser Methode. Die Klasse hat damit mehrere Änderungsgründe (OCP-Verletzung) und wächst unkontrolliert.

7.1.2 Code Smell 2: `PokerCli` als God Class

`PokerCli` vereint zu viele Verantwortlichkeiten in einer Klasse (SRP-Verletzung): Hauptschleife, Command-Dispatching, Ausgabeformatierung für Spielstand, Regeln und Zusammenfassung, Eingabe-Parsing und Fehlerbehandlung. Die Klasse ist dadurch schwer isoliert zu testen und zu erweitern.

7.2 Refactorings

7.2.1 Refactoring 1: Introduce Interface – `RandomSource`

Code Smell: `Deck` und die Use Cases hingen direkt von `java.util.Random` ab. Das machte deterministisches Testen unmöglich und koppelte Domain-Code an eine Java-Standardbibliothek.

Refactoring: Introduce Interface / Dependency Inversion. Das Interface `RandomSource` wurde eingeführt, `JavaRandomSource` kapselt `java.util.Random`, `FixedRandomSource` liefert deterministische Werte für Tests.

Vorher:

```
// Deck.java – direkter Zugriff auf java.util.Random
public void shuffle() {
    Collections.shuffle(cards, new Random());
}
```

Nachher:

```
// RandomSource.java – Interface
public interface RandomSource {
    int nextInt(int upperBoundExclusive);
}
```

```java
// Deck.java – hängt nur noch vom Interface ab
public void shuffle(RandomSource randomSource) {
    for (int i = cards.size() - 1; i > 0; i--) {
        int j = randomSource.nextInt(i + 1);
        Card temp = cards.get(i);
        cards.set(i, cards.get(j));
        cards.set(j, temp);
    }
}
```

Effekt: Domain-Code ist von Java-Standardbibliothek entkoppelt. Tests können `FixedRandomSource` injizieren und damit exakt vorhersagen, welche Karten ausgeteilt werden.

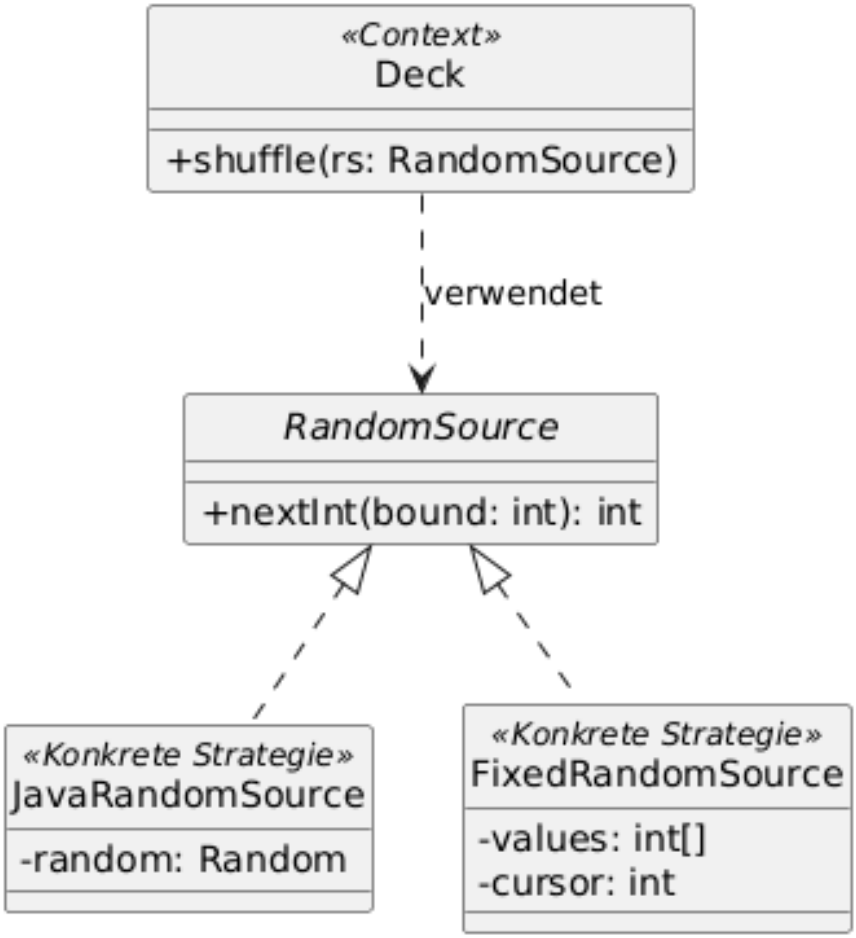

Abbildung 21: Refactoring zu Strategy Pattern: RandomSource Interface mit zwei Implementierungen

7.2.2 Refactoring 2: Extract Class – `GameRules`

Code Smell: Spielkonstanten (Startchips, Blindhöhe, Mindest- und Maximalspielerzahl) waren als Magic Numbers verstreut im Code. Änderungen an Spielregeln erforderten Suche und Anpassung an mehreren Stellen.

Refactoring: Extract Class / Introduce Explaining Variable. Alle Regelkonstanten wurden in `GameRules` zentralisiert.

Vorher (Magic Numbers verstreut):

```java
// StartGameUseCase.java
game.addPlayer(playerName, new Chips(1000));  // Magic Number

// Game.java
if (players.size() < 2) {                     // Magic Number
    throw new IllegalStateException("...");
}
```

Nachher (Single Source of Truth):

```java
// GameRules.java
public final class GameRules {
    public static final int INITIAL_CHIPS = 1_000;
    public static final int SMALL_BLIND   = 10;
    public static final int MIN_PLAYERS   = 2;
    public static final int MAX_PLAYERS   = 9;
    private GameRules() {}
}

// StartGameUseCase.java
game.addPlayer(playerName, new Chips(GameRules.INITIAL_CHIPS));

// Game.java
if (players.size() < GameRules.MIN_PLAYERS) {
    throw new IllegalStateException("...");
}
```

Effekt: Regeländerungen erfordern nur eine Anpassung in `GameRules`. Der Code ist selbstdokumentierend – statt `1000` steht `GameRules.INITIAL_CHIPS`.

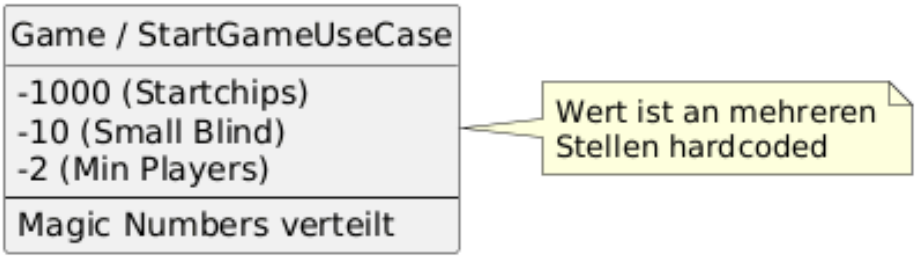

Abbildung 22: Extract Class Refactoring – Vorher: Magic Numbers verstreut

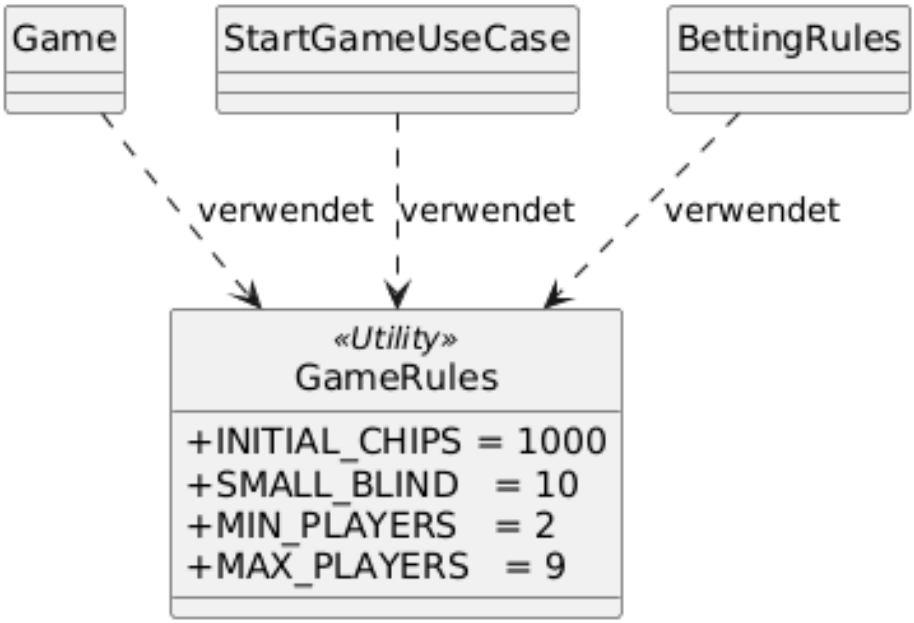

Abbildung 23: Extract Class Refactoring – Nachher: GameRules als Single Source of Truth

7.2.3 Refactoring 3: Move Class – `GameRepository`

Code Smell: Das Interface `GameRepository` lag ursprünglich im `infrastructure`-Paket. Application-Use-Cases importierten es von dort – eine Verletzung der Dependency Rule (äußere Schicht zeigt auf innere).

Refactoring: Move Class / Dependency Inversion. `GameRepository` wurde logisch ins `application`-Paket verschoben, um die korrekte Architektur-Dependency zu etablieren: Infrastructure implementiert, Application definiert.

Vorher (falsche Platzierung):

```
de.pokersim.infrastructure
  └── GameRepository (Interface)  ← Application hängt davon ab

de.pokersim.application
  ├── StartGameUseCase
  └── import de.pokersim.infrastructure.GameRepository  ← ❌ falsche Richtung
```

Nachher (korrekte Platzierung):

```
de.pokersim.application
  ├── GameRepository (Interface)  ← Infrastructure implementiert
  └── StartGameUseCase

de.pokersim.infrastructure
  └── FileGameRepository implements GameRepository  ← ✓ richtige Richtung
```

Effekt: Die Dependency Rule ist korrekt eingehalten: Infrastructure hängt von Application ab, nicht umgekehrt.

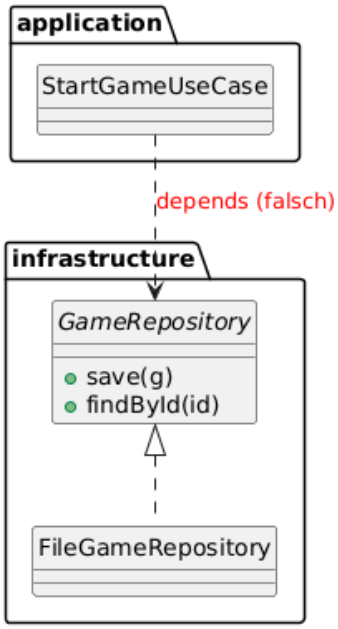

Abbildung 24: Move Class Refactoring – Vorher: Repository im falschen Paket

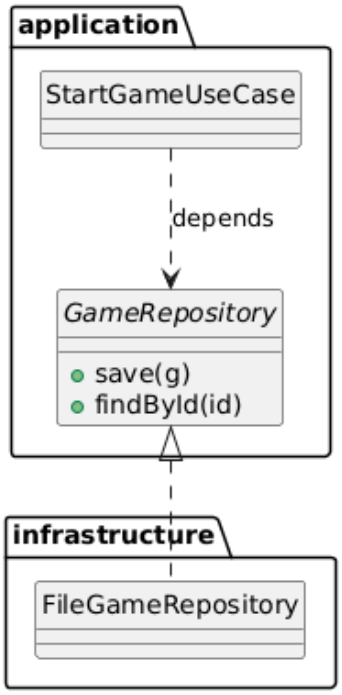

Abbildung 25: Move Class Refactoring – Nachher: Repository an richtigem Ort

8 Entwurfsmuster

8.1 Repository Pattern

8.1.1 Beschreibung

Das **Repository Pattern** kapselt den Datenzugriff hinter einer domänennahen Schnittstelle. Der Domain- und Application-Layer arbeiten ausschließlich mit dem Interface und kennen weder Datenbankdetails noch Serialisierungslogik. Implementierungen können ausgetauscht werden, ohne dass Aufrufer geändert werden müssen.

8.1.2 Umsetzung in PokerSim

```
        «interface»
       GameRepository
      ┌──────────────┐
      │ +save(Game)  │
      │ +findById()  │
      └──────────────┘
             │
             │ implements
    ┌────────┼────────────────┐
    │        │                │
    ▼        ▼                ▼
InMemory  FileGame       FakeGame
GameRepo  Repository     Repository
```

```java
// Interface
public interface GameRepository {
    void save(Game game);
    Optional<Game> findById(GameId id);
}

// Produktions-Implementierung: schreibt auf Disk
public final class FileGameRepository implements GameRepository {
    private final Path storageFile;
    // ...
    @Override
    public void save(Game game) {
        GameSnapshot snapshot = snapshotMapper.toSnapshot(game);
        String serialized = serializer.serialize(snapshot);
        Files.writeString(storageFile, serialized, StandardCharsets.UTF_8);
    }
}

// Test-Implementierung: In-Memory
public final class InMemoryGameRepository implements GameRepository {
    private final Map<GameId, Game> games = new HashMap<>();
    @Override
    public void save(Game game) { games.put(game.id(), game); }
    @Override
    public Optional<Game> findById(GameId id) {
        return Optional.ofNullable(games.get(id));
    }
}
```

8.1.3 Begründung

- Die `StartGameUseCase` und alle anderen Use Cases hängen ausschließlich vom Interface ab – nicht von einer konkreten Implementierung.

- Im Produktionsbetrieb wird `FileGameRepository` via Konstruktor-Injection in `Main.java` eingesetzt.
- In Tests wird `FakeGameRepository` oder `InMemoryGameRepository` eingesetzt – ohne Änderung an den Use Cases.
- Das Muster unterstützt die Dependency Rule: Der Application-Layer definiert, was er braucht (`GameRepository`), Infrastructure liefert die Implementierung.

8.2 Strategy Pattern

8.2.1 Beschreibung

Das **Strategy Pattern** kapselt eine Familie von Algorithmen, macht sie austauschbar und entkoppelt den Algorithmus vom Aufrufer. Der Aufrufer kennt nur das Interface – nicht die konkrete Strategie.

8.2.2 Umsetzung in PokerSim

```
     «interface»
    RandomSource
   ┌──────────────────────────┐
   │ +nextInt(bound): int     │
   └──────────────────────────┘
                 │
                 │ implements
       ┌─────────┴──────────┐
       │                    │
       ▼                    ▼
JavaRandomSource     FixedRandomSource
(Produktion)            (Tests)
```

```java
// Interface – Strategy
public interface RandomSource {
    int nextInt(int upperBoundExclusive);
}

// Konkrete Strategie: echter Java-Zufall
public final class JavaRandomSource implements RandomSource {
    private final Random random;
    public JavaRandomSource() { this.random = new Random(); }
    public JavaRandomSource(long seed) { this.random = new Random(seed); }

    @Override
    public int nextInt(int upperBoundExclusive) {
        return random.nextInt(upperBoundExclusive);
    }
}

// Konkrete Strategie: deterministisch für Tests
public final class FixedRandomSource implements RandomSource {
    private final int[] values;
    private int cursor;

    public static FixedRandomSource noShuffle() { return new
FixedRandomSource(0); }

    @Override
    public int nextInt(int upperBoundExclusive) {
        int raw = values[cursor % values.length];
        cursor++;
```

```java
        return Math.floorMod(raw, upperBoundExclusive);
    }
}

// Aufrufer – kennt nur das Interface
public final class Deck {
    public void shuffle(RandomSource randomSource) {
        for (int i = cards.size() - 1; i > 0; i--) {
            int j = randomSource.nextInt(i + 1);
            // Fisher-Yates-Shuffle
        }
    }
}
```

8.2.3 Begründung

- `Deck.shuffle()` und `StartGameUseCase` nehmen `RandomSource` entgegen – nicht `JavaRandomSource`. Die konkrete Strategie wird von außen per Konstruktor-Injection übergeben.
- In Produktion wird `JavaRandomSource` über `Main.java` injiziert.
- In Tests wird `FixedRandomSource.noShuffle()` oder eine vordefinierte Sequenz eingesetzt, was deterministische Kartenreihenfolgen und reproduzierbare Tests ermöglicht.
- Das Muster ist direkt mit dem Prinzip **Geringe Kopplung** (GRASP) und dem **Dependency Inversion Principle** (SOLID) verzahnt.

Das Strategy Pattern ist auch für `HandEvaluator` ⟶ `TexasHoldemHandEvaluator` realisiert: Die Hand-Bewertungsstrategie kann ohne Änderung an `Game` oder `ShowdownUseCase` ausgetauscht werden (z. B. für Omaha-Hold'em).

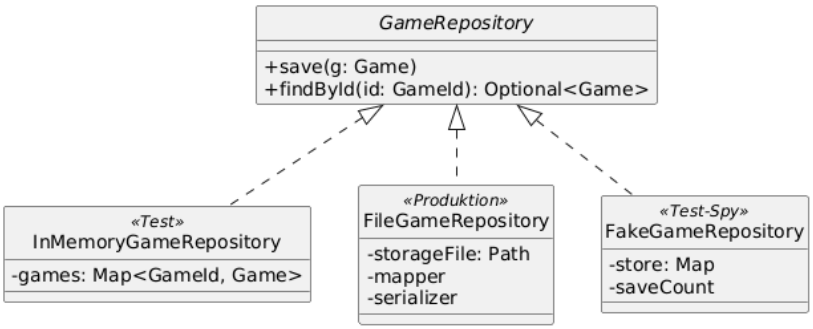

Abbildung 26: Repository Pattern: `GameRepository` mit drei Implementierungen

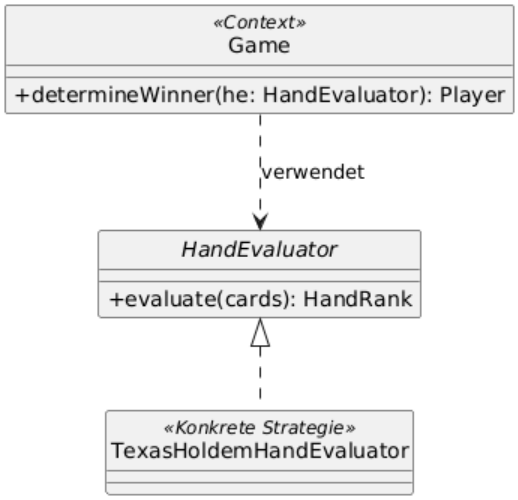

Abbildung 27: Strategy Pattern für Hand-Bewertung: austauschbare Strategien